Resumo do projeto

Verificação adversarial com 3 agentes para reduzir falsos positivos na detecção de vulnerabilidades. Auditoria de Belozerov et al. (EMSE, 2026) e teste de um pipeline com modelos abertos. Material de estudo do grupo: explica o projeto inteiro, do começo ao fim. Projeto da disciplina TAES, Centro de Informática da UFPE, novembro de 2026.

1. O projeto em uma página

De onde partimos. Um artigo de 2026 testou se LLMs comerciais (GPT-4.1, GPT-5, Claude Opus 4.1) conseguem detectar e corrigir vulnerabilidades em código C, C++ e C#, e divulgou cerca de 80% de detecção e 77% de correção. Mas o artigo só mede quantas das vulnerabilidades conhecidas o modelo encontra. Ele nunca mede quantos **alarmes falsos** o modelo dá, embora tivesse os dados para isso.

O que fizemos.

1. **Auditamos o artigo:** recalculamos as tabelas a partir dos dados publicados pelos autores, sem usar LLM.
2. **Testamos uma proposta para reduzir falsos positivos:** um pipeline com três papéis, em que um agente detecta, outro corrige e um terceiro tenta **refutar** cada achado (o verificador adversarial). Rodamos tudo com modelos abertos e gratuitos, em GPUs gratuitas do Kaggle, com custo financeiro zero.
3. **Medimos precisão e recall** usando só os rótulos que os próprios autores produziram: 56 issues reais e 85 falsos alarmes conhecidos. Não houve anotador humano.

O que encontramos.

- O verificador removeu **85%** dos falsos alarmes conhecidos que o detector acusava, mas também **64%** das issues reais. A taxa de falsos alarmes caiu de 15,3% para 2,4%, e o recall caiu de 19,6% para 7,1%.
- A hipótese principal (reduzir falsos alarmes perdendo no máximo 5 pontos de recall) **não se confirmou**, nem na execução principal nem em 3 repetições.
- O verificador **não separa** real de falso de forma significativa nos achados do detector. Ele só muda o **ponto de operação**: menos alarmes e menos detecção.
- A auditoria mostrou que a "acurácia" do artigo é, na verdade, recall relativo, e encontrou erros de tabela, afirmações que não se sustentam e uma "melhora" entre versões que não é estatisticamente significativa.

A frase para lembrar: o verificador adversarial corta falsos alarmes, mas corta issues reais junto; ele desloca o ponto de operação em vez de melhorar a discriminação.

2. O artigo analisado

Belozerov, Barclay e Sami, *Secure coding with AI – from detection to repair*, Empirical Software Engineering 31:93 (2026). O processo do artigo foi:

1. Extrair 2.315 trechos de código C, C++ e C# do **DevGPT**, um conjunto público de conversas reais de desenvolvedores com o ChatGPT (snapshots v8, de outubro de 2023, e v10, de maio de 2024).
2. Passar esses trechos por **quatro scanners estáticos**: Snyk, Semgrep, Flawfinder e Cppcheck. Daí saíram **114 arquivos**.
3. Revisar à mão os alertas: **56 vulnerabilidades reais em 48 arquivos**. Os demais alertas (85 locais) foram julgados seguros, e 66 arquivos ficaram sem nenhuma vulnerabilidade real.
4. Pedir a GPT-4.1, GPT-5 e Claude Opus 4.1, com **um único prompt** ("You must find security issues..."), que achassem e corrigissem os problemas.
5. Contar quantas das 56 foram encontradas e corrigidas: ~80% e ~77%.

O problema de método. A conta só olha para as 56. Um modelo que acusasse tudo, em todos os arquivos, teria "acurácia" de 100%. Os dados para medir falso positivo existiam: os 66 arquivos negativos e os 85 locais julgados seguros. O prompt ainda empurra o modelo a sempre encontrar algo.

O que os próprios dados do artigo mostram. Recontamos as CWEs citadas nas 345 respostas originais dos modelos:

- os modelos citam de **4,8 a 6,5 CWEs por arquivo**, contra cerca de 1,2 issue real por arquivo positivo (56 ÷ 48);
- **98,5% a 100%** dos arquivos negativos recebem pelo menos uma CWE;
- dos 85 falsos alarmes conhecidos, de **35% a 42%** são citados pela leitura mais estrita (de 67% a 72% pela mais leniente);
- cada modelo rodou **uma única vez**, sem intervalo de confiança.

3. A auditoria do artigo

Como foi feita. Tudo foi recalculado com código (`pipeline/src/f2_audit.py`), sem LLM, a partir de duas fontes: as tabelas do PDF do artigo (lidas automaticamente, com uma linha por issue e "Yes/No" de detecção e correção para cada modelo) e o repositório público dos autores (planilha de rótulos, saídas dos scanners, respostas dos modelos e configuração). Cada afirmação do texto foi conferida contra a recontagem, com a página citada.

Item	O que encontramos	Veredito
Tabela 11 (GPT-4o, 2024)	O texto diz 17 corrigidas e 15 não; a tabela tem 16 e 16	erro confirmado
Tabelas 17 e 18 (código do usuário × gerado)	3 de 6 células divergem da recontagem	erro confirmado
Finding 15: "todos vão melhor em código do usuário"	Não vale para GPT-4.1 nem para Opus 4.1	refutado
"GPT-5 corrigiu todas as 44 que detectou"	Detectou 44 e corrigiu 44, mas não as mesmas	refutado
Gabarito de 2024 × 2025 (mesmo snapshot)	4 issues saíram; uma mudou de origem	confirmado
"Melhora" de 2024 para 2025	McNemar exato nas 28 issues comuns: p = 0,065 a 0,267	não significativa
Reescaneamento do código corrigido	Está no resumo, mas no fim do artigo aparece como trabalho futuro	inconsistência

Item	O que encontramos	Veredito
Número de scanners e critério de seleção	O texto diz 3 scanners (são 4); 49 das 56 issues vêm de um só scanner, embora o critério fale em "vários scanners na mesma linha"	inconsistência
"Accuracy" nos Findings 4 e 15	É recall relativo, sem precisão nem falso positivo	rótulo inadequado
CWEs na Tabela 14	As CWEs de `v10:3/7` e `v10:3/11` estão trocadas	erro confirmado

Como ler isso. A auditoria não invalida o artigo. Ela mostra que a métrica tem o nome errado, que o falso positivo não foi medido e que algumas afirmações não se sustentam na recontagem. O conjunto de dados que os autores publicaram foi justamente o que tornou o nosso trabalho possível.

4. A pergunta e as hipóteses

Pergunta: um verificador adversarial reduz falsos positivos **sem perder detecção**?

As hipóteses foram fixadas antes de rodar qualquer coisa:

ID	Hipótese	Comparação
H1 (principal)	O verificador reduz a taxa de falsos alarmes conhecidos (e aumenta a precisão) perdendo no máximo 5 p.p. de recall	C3 × B1
H2	Parte da redução vem só do prompt neutro, que aceita "nenhuma vulnerabilidade"	B1 × B0
H3	Verificar a falha sem ver a correção rejeita mais falsos alarmes do que verificar com a correção à vista	C3 × C2
H4	O verificador vale mais que um simples corte por confiança do detector	C1/C3 × curva do B1
H5	Um verificador de outra família remove mais falsos positivos que um da mesma família	C3 × C3-mf
E1 (exploratória)	O pipeline acusa menos falsos alarmes que os modelos do artigo	C3 × respostas do artigo

H1 a H3 são **confirmatórias**: foram as comparações planejadas como principais, com teste estatístico e correção para comparações múltiplas. O resto é **exploratório**, ou seja, serve para entender os dados, não para provar hipóteses.

5. A proposta: três papéis

Papel	O que faz	Modelo
A1, detector	Lê o código com linhas numeradas e devolve achados estruturados: CWE, linhas, de onde vem o dado do atacante (fonte), onde ele chega (*sink*), o gatilho e uma confiança. "Nenhuma vulnerabilidade" é resposta aceitável.	qwen2.5-coder 14B
A2, corretor	Escreve um patch mínimo para um achado e explica como o gatilho passa a ser bloqueado.	qwen2.5-coder 14B

Papel	O que faz	Modelo
A3a, verificador do achado	Recebe **um** achado por vez e tenta refutá-lo. O veredito padrão é "refutado".	gpt-oss 20B
A3 conjunto	Mesmo prompt do A3a, mas recebe o achado e o patch juntos (usado só no C2).	gpt-oss 20B
A3b, verificador da correção	Julga se o patch corrige o problema, com ajuda de compilação e scanners.	gpt-oss 20B

Quando o A3a confirma. Só se as três condições forem verdadeiras: (1) existe uma fonte de dados não confiáveis plausível (entrada do usuário, arquivo, rede, argumentos, variável de ambiente, parâmetro de API pública); (2) existe um caminho da fonte até a operação perigosa que o código não neutraliza; (3) existe um gatilho concreto. Antes de confirmar, ele precisa checar se há validação em outra linha, se os argumentos são constantes, se o caminho é inalcançável e se o item é só qualidade de código.

Por que o A3b tem ferramentas e os outros não. O gabarito foi construído a partir dos mesmos scanners. Se o detector ou o verificador vissem a saída dos scanners, receberiam a lista dos locais candidatos ao gabarito, e o recall subiria artificialmente. No A3b isso não é problema: a decisão de detecção já foi tomada, e ele só avalia o patch. O A3b recebe a compilação só de sintaxe (`-fsyntax-only`) do original e do patch, e o reescaneamento com flawfinder, cppcheck e semgrep, que mostra se o alerta sumiu e se apareceu algum novo. O código dos autores nunca é executado.

Outras regras de desenho. O verificador e o corretor nunca veem a confiança do detector nem os rótulos. O verificador é de outra família de modelos que o detector (gpt-oss, da OpenAI, contra qwen, da Alibaba), para reduzir erros em comum.

6. Desenho experimental: seis configurações

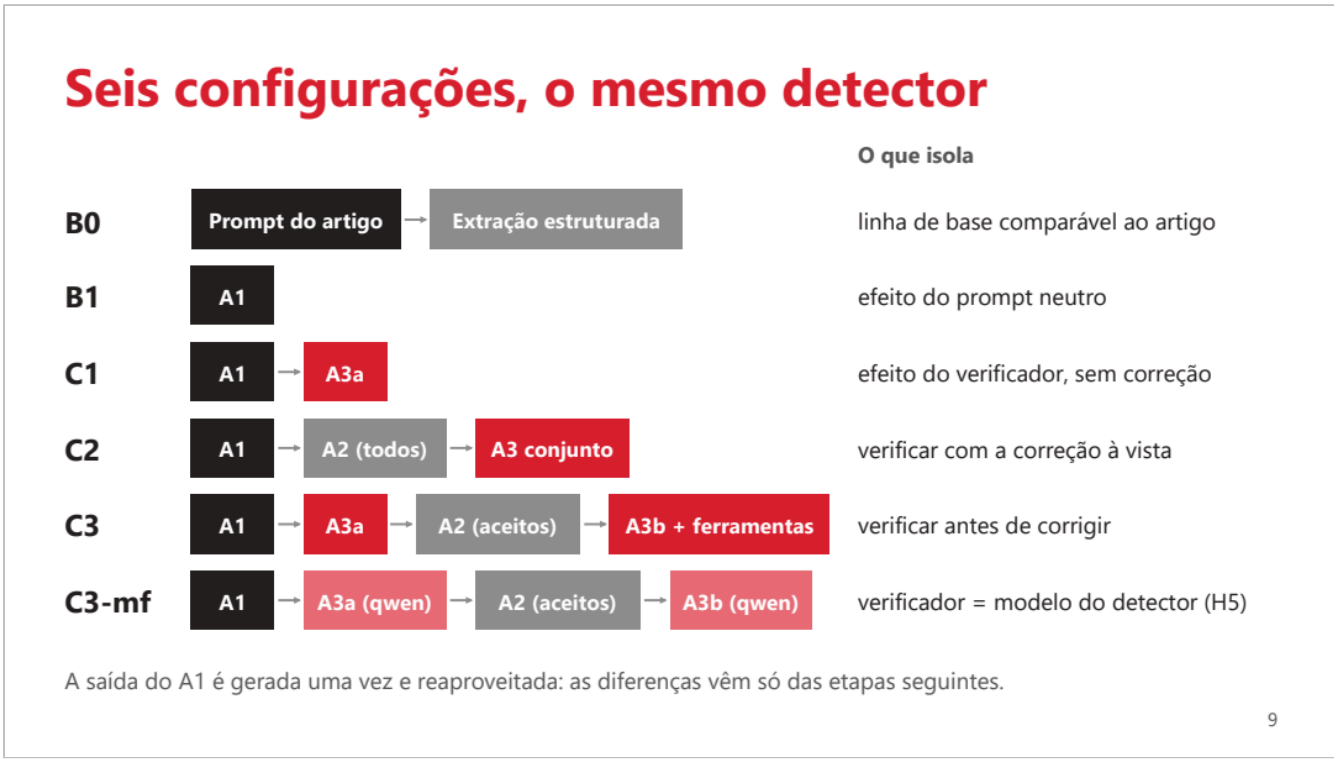


Figura 1. As seis configurações. Preto: detector (ou o prompt do artigo); cinza: corretor ou extrator; vermelho: verificador; rosa: verificador trocado pelo próprio qwen.

A ideia é a de um experimento de laboratório: cada configuração acrescenta ou troca **uma** etapa em relação a outra, então a diferença de resultado entre as duas mostra o efeito daquela etapa. Exceto no B0, o A1 roda **uma vez** por arquivo, e a mesma lista de achados é usada por todas as outras configurações.

- **B0, "como no artigo":** o prompt original do artigo no nosso modelo. A resposta vem em texto livre, e um extrator (o mesmo modelo) a converte em achados estruturados, para aplicar a mesma regra de contagem.
- **B1, só o detector:** o A1 com prompt neutro. Tudo o que ele acusa conta.
- **C1, detector + verificador:** cada achado vai para o A3a; só os confirmados contam.
- **C2, corrige e depois verifica:** o A2 escreve um patch para **todos** os achados, e o A3 conjunto julga cada achado já vendo o patch. Era a ordem da proposta original.
- **C3, verifica e depois corrige:** o A3a filtra; o A2 corrige só os confirmados; o A3b julga cada correção.
- **C3-mf:** igual ao C3, mas o verificador e o juiz da correção são o próprio qwen.

Exemplo. Suponha que o A1 acuse três achados num arquivo: (a) um `strcpy` sem checar tamanho, que é uma issue real; (b) um `printf` com formato constante, que é um falso alarme conhecido; (c) uma "injeção de comando" que não está no gabarito. O B1 acusa os três. O C1 manda cada um ao A3a; se ele confirmar só o (a), o C1 acusa só o (a). O C2 escreve três patches, e o A3 conjunto decide vendo achado e patch. O C3 confirma o (a), corrige só ele e julga o patch.

Duas consequências importantes. (1) **C1 e C3 têm sempre o mesmo resultado de detecção**: nos dois quem decide é o A3a, e as etapas a mais do C3 não removem achados. Eles só diferem na correção e no custo. (2) O C2 poderia ter dado diferente (o receio era que um patch bem escrito convencesse o verificador de que a falha era real), mas acusou os mesmos 2 falsos alarmes e o mesmo número de issues reais que o C3.

7. Dados, gabarito e como contamos

7.1 O gabarito

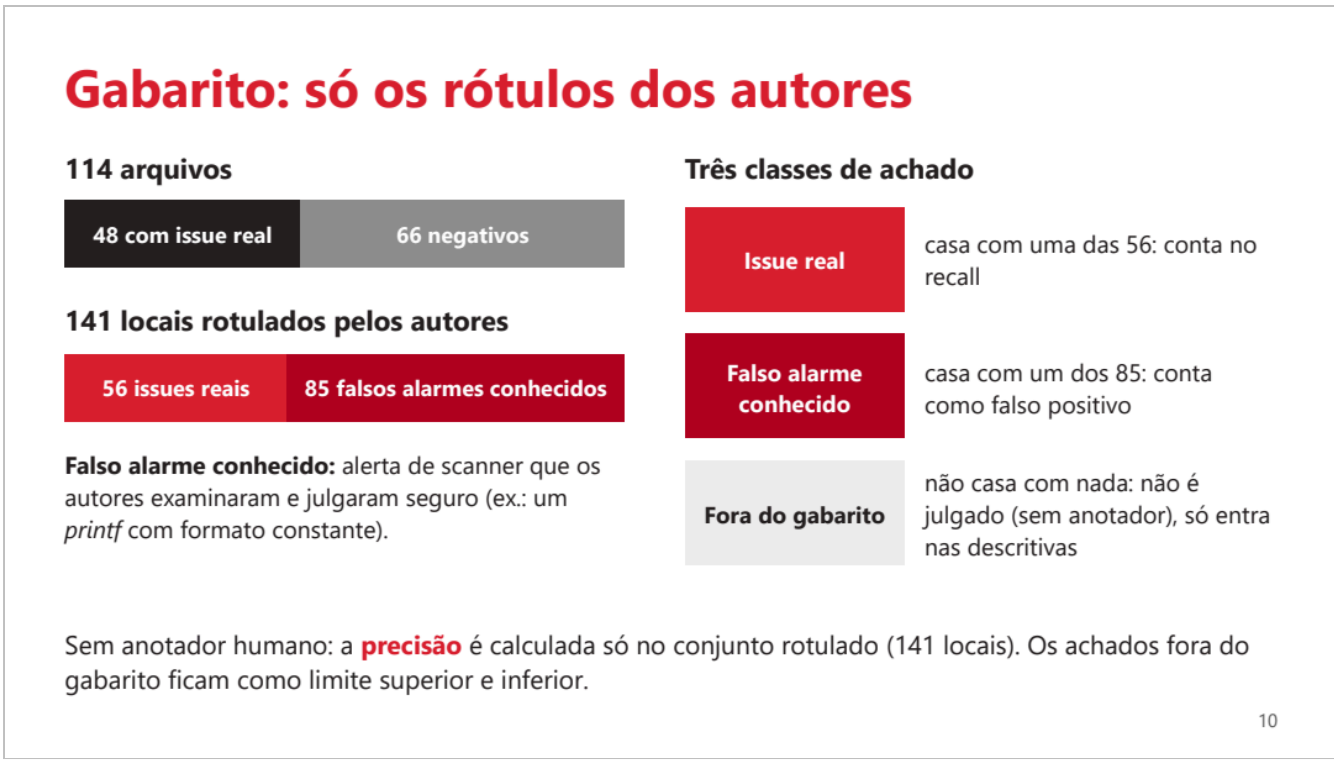


Figura 2. O gabarito usado: só os rótulos dos autores.

- **114 arquivos:** 48 com pelo menos uma issue real e 66 negativos.
- **141 locais rotulados:** 56 **issues reais** e 85 **falsos alarmes conhecidos** (alertas de scanner que os autores examinaram e julgaram seguros).
- Cada achado aceito cai em uma de três classes: casa com uma issue real (conta no recall), casa com um falso alarme conhecido (conta como falso positivo) ou fica **fora do gabarito**. Esta última classe não é julgada: pode ser uma alucinação ou uma vulnerabilidade real que os scanners não viram.

7.2 A regra de correspondência

Um achado casa com um local do gabarito se estiver no **mesmo arquivo**, a **até 3 linhas** de distância e com **CWE compatível**: a mesma CWE, uma ancestral ou descendente na hierarquia oficial da MITRE (*CWE Research View 1000*), ou da mesma família. Cada issue conta no máximo uma vez. Essa é a regra **estrita**. Como ela pesa muito, fizemos sensibilidades; a principal é a **S1**, que ignora a CWE e olha só a linha.

7.3 Métricas, com um exemplo

Para o B1, no conjunto rotulado:

	Acusado	Não acusado
Issue real (56)	11	45
Falso alarme conhecido (85)	13	72

- **Recall** = 11 ÷ 56 = **19,6%**: das issues reais, quantas são acusadas.
- **Taxa de falsos alarmes** = 13 ÷ 85 = **15,3%**: dos locais seguros, quantos são acusados por engano.
- **Precisão** = 11 ÷ (11 + 13) = **45,8%**: dos acusados rotulados, quantos são reais.
- **F1** (média harmônica de precisão e recall) = **27,5%**. O **F0,5** dá mais peso à precisão: 36,2%.
- **MCC** vai de −1 a +1, e 0 equivale a acusar ao acaso. No B1 dá 0,06; no C3, 0,12. Os dois estão perto de zero.
- **Por arquivo:** a fração de **arquivos negativos sinalizados** é um limite superior da taxa de falso positivo por arquivo.

7.4 Estatística

- **Intervalos de 95% por bootstrap sobre arquivos:** reamostramos os 114 arquivos 2.000 vezes. Reamostrar arquivos inteiros respeita o fato de que achados do mesmo arquivo não são independentes.
- **Diferenças pareadas:** em cada réplica, a mesma amostra de arquivos entra nas duas configurações. Se o intervalo da diferença não inclui 0, a diferença é significativa.
- **McNemar exato:** compara duas configurações olhando só os locais em que elas discordam. Exemplo: issues reais acusadas só pelo B1 = 7, só pelo C3 = 0; a chance de 7 contra 0 por acaso é $2 \times 0{,}5^7 \approx 0{,}016$.
- **Q de Cochran** (cinco configurações ao mesmo tempo), **correção de Holm** (comparações múltiplas), **Fisher exato** (tabelas 2 × 2 com contagens pequenas) e **κ de Cohen** (concordância descontando o acaso; abaixo de 0,4 é fraca).

8. Infraestrutura e execução

- **Modelos:** `qwen2.5-coder:14b` (quantizado em Q4_K_M) como detector e corretor; `gpt-oss:20b` (MXFP4, raciocínio médio) como verificador. Ambos servidos pelo Ollama, em duas GPUs T4 gratuitas do Kaggle. O digest de cada modelo foi conferido em toda execução.
- **Controle:** temperatura 0 e seed 0 na execução principal; até 4 achados por arquivo; achados repetidos removidos. Os prompts foram **congelados com hash** antes da execução principal, e cada decisão foi registrada num diário.
- **Cache:** cada chamada fica guardada por uma chave que inclui modelo, prompt, parâmetros e entrada. Isso permitiu reaproveitar exatamente os mesmos achados do A1 em todas as configurações.
- **Piloto:** em 10 arquivos, a primeira versão do prompt do detector quase não achava issues reais (0 de 9). Ela foi ajustada uma vez, com critérios fixados antes e **sem olhar o gabarito** (pelo menos 2 achados por arquivo, pelo menos 3 CWEs diferentes, nenhuma falha de formato). Para não "ajustar no teste", a sensibilidade S4 refaz a análise só nos outros 104 arquivos.
- **Calibração do verificador:** antes da execução principal, o A3a julgou os 141 locais rotulados (reconstruídos a partir das saídas dos scanners). Ele confirmou 39% dos reais e 5% dos falsos ($p = 3 \times 10^{-7}$). Ou seja, sobre achados de scanner, ele discrimina bem.
- **Execuções:** principal (114 arquivos, 5 configurações, 9,7 h de GPU em duas metades), variância (B1 e C3 com seeds 1 a 3, 3,94 h), extração das respostas do artigo nos negativos (1,03 h), ablação C3-mf (0,76 h) e A3b nos patches do B0 (2,17 h). Total: **17,6 h de GPU**, mais ~3,4 h de piloto e ajustes. Custo financeiro: zero.

9. Resultados

9.1 Resultado principal

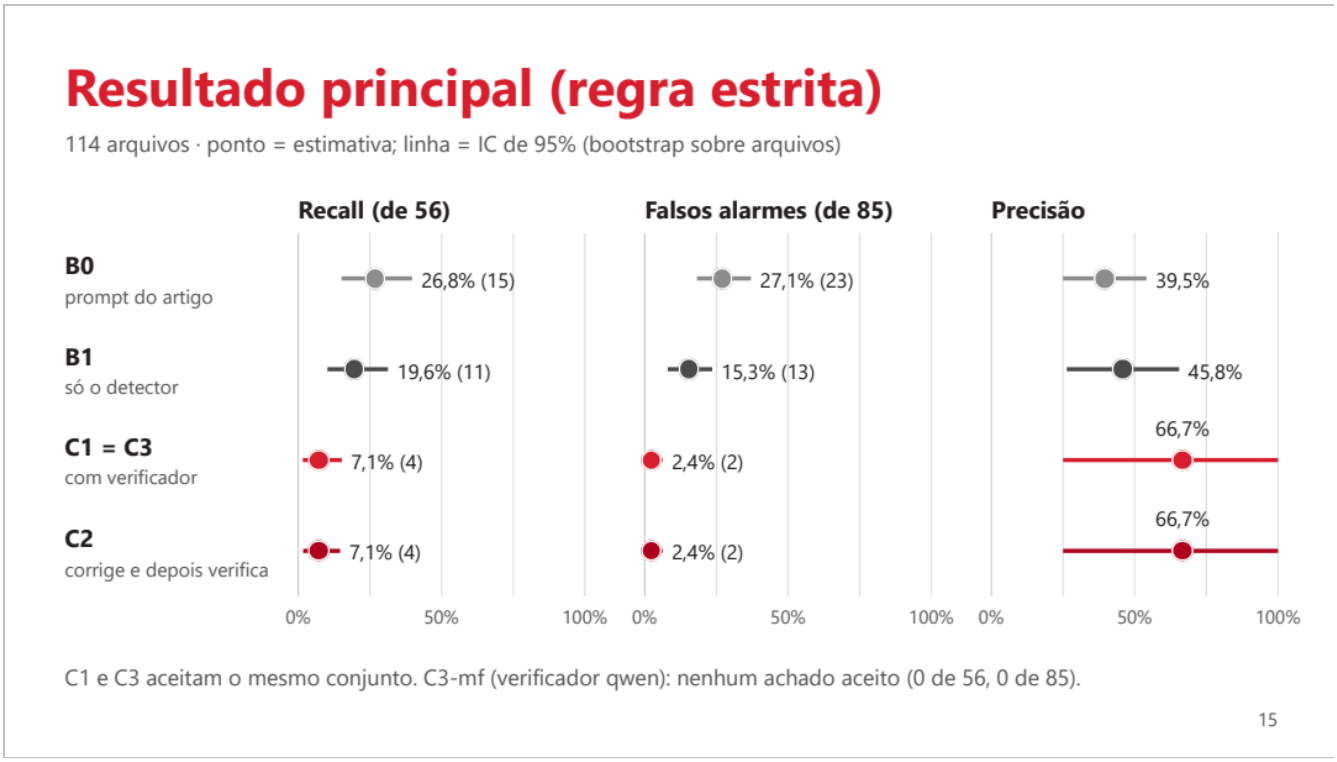


Figura 3. Recall, taxa de falsos alarmes e precisão por configuração (regra estrita; ponto = estimativa, linha = IC de 95%).

Config	Recall	Falsos alarmes	Precisão	F1	MCC
B0	26,8% (15/56)	27,1% (23/85)	39,5%	31,9%	−0,00
B1	19,6% (11/56)	15,3% (13/85)	45,8%	27,5%	0,06
C1 = C3	7,1% (4/56)	2,4% (2/85)	66,7% [25; 100]	12,9%	0,12
C2	7,1% (4/56)	2,4% (2/85)	66,7% [25; 100]	12,9%	0,12
C3-mf	0% (0/56)	0% (0/85)	indefinida	0%	indefinido

A precisão do C3 sobe para 66,7%, mas se apoia em 6 locais (4 reais e 2 falsos) e o intervalo vai de 25% a 100%.

9.2 O que o verificador faz

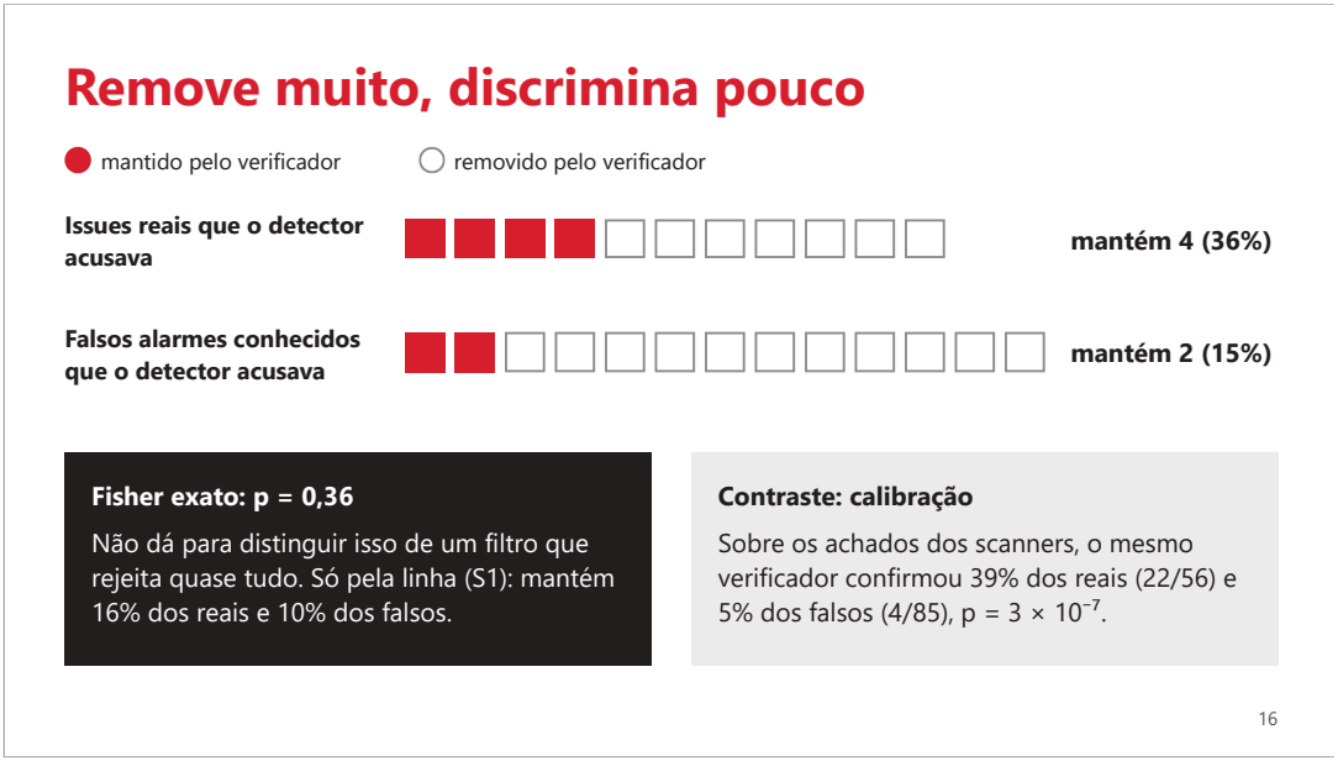


Figura 4. O que o verificador fez com os locais rotulados que o detector acusava.

Dos locais rotulados que o B1 acusava, o verificador removeu **11 de 13 falsos alarmes (85%)** e **7 de 11 issues reais (64%)**. Ele manteve 36% dos reais e 15% dos falsos. Proporcionalmente, manteve mais reais, mas com contagens tão pequenas o teste de Fisher dá **p = 0,36**: não dá para distinguir isso de um filtro que rejeita quase tudo. Pela regra só-linha (S1), o padrão se repete: mantém 16% dos reais e 10% dos falsos.

O contraste. Na calibração, sobre os achados dos scanners, o mesmo verificador discriminou bem (39% × 5%). Sobre os achados do nosso detector, não. A capacidade de separar real de falso depende do tipo de achado que o verificador recebe. Uma explicação possível, não testada: os achados de scanner trazem padrões que o prompt de refutação manda checar (argumento constante, validação em outra linha), enquanto os achados do detector já chegam com uma narrativa de fonte até a operação perigosa.

9.3 As hipóteses

Hipótese	O que medimos (diferença A − B, IC de 95%)	Veredito
H1 (C3 − B1)	Recall −12,5 p.p. [−22,0; −4,7]; falsos alarmes −12,9 [−20,5; −6,1]; precisão +20,8 [−18,4; +58,6]	**Não confirmada:** o recall cai bem mais que 5 p.p. e a precisão não melhora de forma significativa
H2 (B1 − B0)	Falsos alarmes −11,8 [−20,2; −3,7]; recall −7,1 [−20,0; +4,9]; McNemar p = 0,013 (Holm 0,052)	**Apoiada, com ressalva:** só o prompt neutro já reduz os falsos alarmes; mas 28% dos achados do B0 não têm linha e não casam pela regra estrita
H3 (C3 − C2)	Diferença zero: os dois acusam os mesmos 2 falsos alarmes	**Não confirmada.** A vantagem do C3 é de custo: 2 patches desnecessários contra 14, e 72 s contra 197 s por arquivo
H4	No mesmo nível de falsos alarmes do C3 (2,4%), o melhor corte de confiança do B1 acusa 1 de 56; o verificador acusa 4 de 56	**Evidência fraca** a favor (+5,4 p.p., com base em 4 contra 1 locais)
H5 (C3 − C3-mf)	O qwen como verificador refutou os 229 achados	**Não apoiada, e o teste é degenerado** (seção 9.8)

9.4 Verificador × corte de confiança (H4)

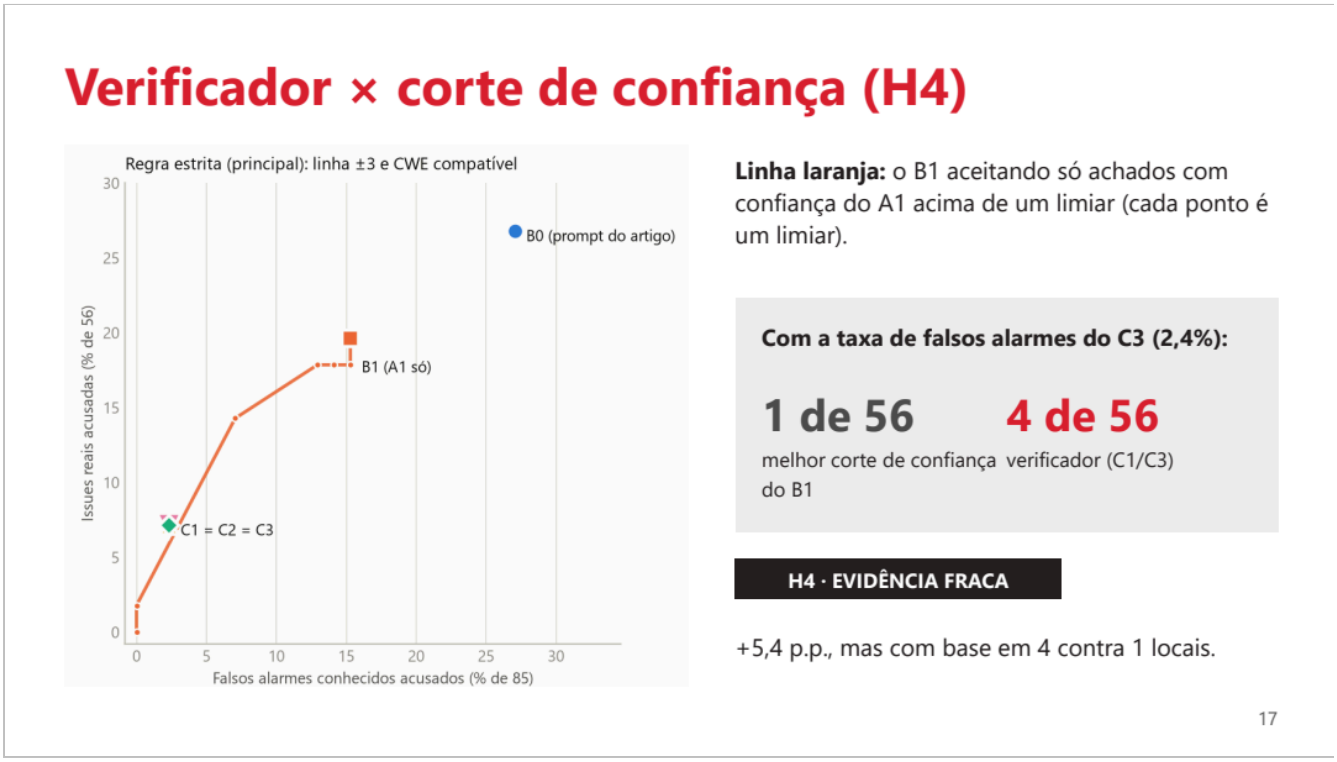


Figura 5. Curva do B1 variando o corte de confiança do detector e o ponto do C3.

A curva laranja é o que se consegue sem verificador, só exigindo mais confiança do detector; cada ponto é um limiar. Se o verificador fosse um "corte de confiança disfarçado", o C3 cairia sobre a curva. Ele fica um pouco acima, mas com 4 contra 1 locais a evidência é fraca.

9.5 Variância: a conclusão se repete

A execução principal é única. Por isso repetimos B1 e C3 três vezes com amostragem (temperatura 0,7 no detector e 1,0 no verificador, seeds 1 a 3):

Repetição	B1 recall	B1 falsos alarmes	C3 recall	C3 falsos alarmes	C3 precisão
var1	12,5%	14,1%	5,4%	2,4%	60,0%
var2	14,3%	17,6%	1,8%	4,7%	20,0%
var3	17,9%	15,3%	7,1%	3,5%	57,1%
Principal (T = 0)	19,6%	15,3%	7,1%	2,4%	66,7%

O critério da H1 foi cumprido em **0 de 3** repetições. A queda de falsos alarmes é consistente (−11,8 a −12,9 p.p., intervalo abaixo de zero nas três), e a perda de recall também (−7,1 a −12,5 p.p.). A precisão do C3 oscila de 20% a 60%.

9.6 O que se ganha e o que se paga

- **Ruído:** arquivos negativos sinalizados caem de 93,9% (B0) e 54,5% (B1) para **9,1%** (C3). Os achados fora do gabarito caem de 328 (B0) e 202 (B1) para **16** (C3). Não sabemos quantos dos removidos eram vulnerabilidades reais que os scanners não viram.
- **Custo:** tokens por issue real acusada: 16 mil no B1, 44 mil no B0, 188 mil no C3 (cerca de 12 vezes o B1) e 344 mil no C2 (cerca de 21 vezes). Segundos de chamada por arquivo: B1 23, C1 54, C3 72, B0 158, C2 197.

9.7 Contra os modelos do artigo (E1) e o peso do modelo

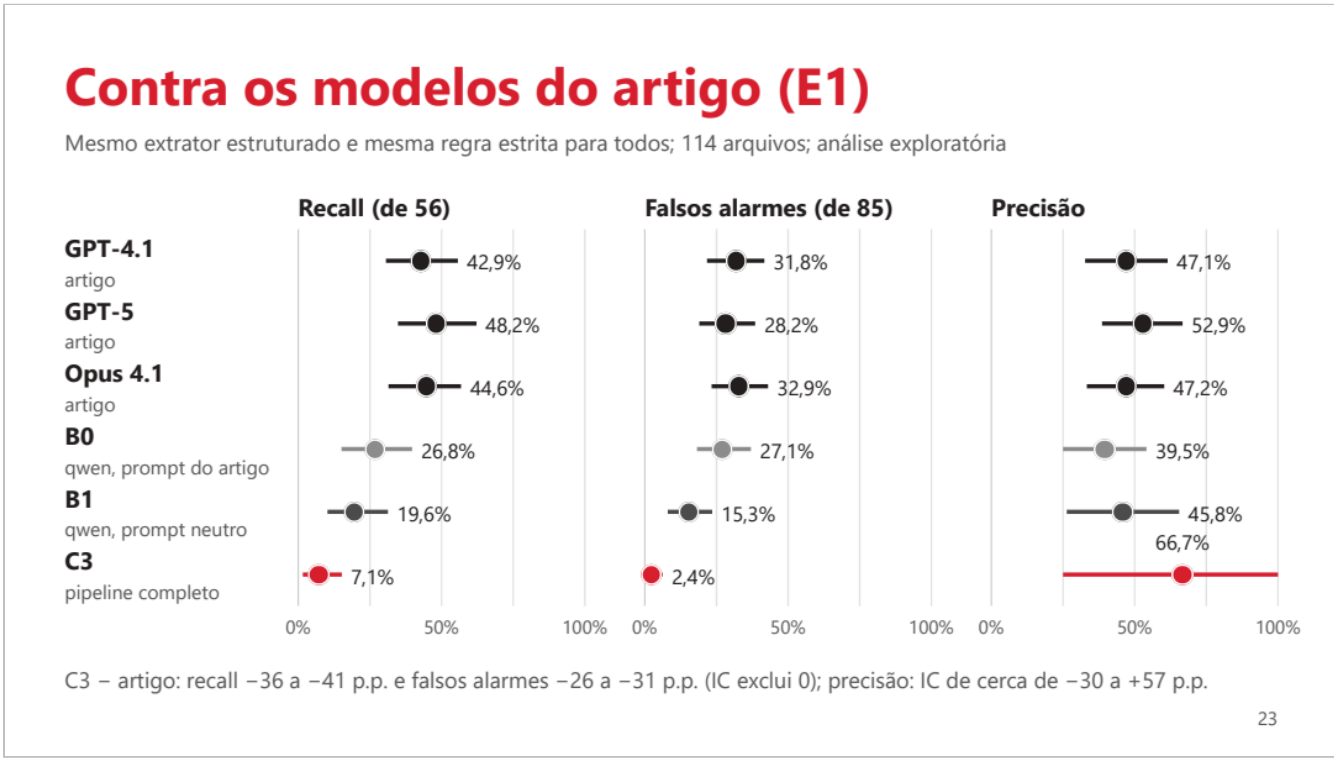


Figura 6. Comparação com as respostas originais dos modelos do artigo, pelo mesmo extrator e pela mesma regra.

As respostas originais de GPT-4.1, GPT-5 e Opus 4.1 passaram pelo mesmo extrator e pela mesma regra. Eles acusam de **43% a 48%** das issues reais e de **28% a 33%** dos falsos alarmes; o C3, 7% e 2%. A diferença do C3 é de −36 a −41 p.p. em recall e −26 a −31 p.p. em falsos alarmes, mas a precisão não se distingue (intervalo de cerca de −30 a +57 p.p.). É outro ponto de operação, não um pipeline melhor. Pela regra só-linha, os modelos do artigo chegam a 84%–91% de recall, perto dos ~80% divulgados, o que mostra que a reconstrução está coerente com o artigo.

O que muda ao trocar o modelo. Com o mesmo prompt, o mesmo extrator e a mesma regra, o qwen de 14B (B0) acusa **16 a 21 p.p. menos** issues reais que os modelos do artigo, com taxa de falsos alarmes parecida (diferença de −1 a −6 p.p., intervalo incluindo 0). Olhando só a linha, os modelos maiores apontam mais de tudo, reais e falsos, com a mesma precisão (+0,5 a +2,8 p.p.). Conclusões:

1. Os números absolutos dependem muito do modelo; o recall baixo é, em boa parte, efeito do detector pequeno.
2. O modelo maior não resolve o falso positivo.
3. O efeito do verificador depende de quem verifica: o mesmo prompt deu 10% de confirmações no gpt-oss e 0% no qwen.
4. Em nenhuma troca o verificador separou melhor o real do falso.

A H1 continua válida, porque compara configurações com o mesmo detector e o mesmo verificador. Essa comparação de modelos é exploratória e foi feita depois dos resultados.

9.8 H5: o verificador do mesmo modelo refuta tudo

No C3-mf, o verificador é o próprio qwen. Ele refutou **os 229 achados**, com confiança de 0,8 a 1,0 e argumentos plausíveis; o gpt-oss tinha confirmado 23 (10%). Então o C3-mf acusa 0 issues reais e 0 falsos alarmes. Formalmente a H5 não é apoiada (falsos alarmes C3 − C3-mf: +2,4 p.p. [0,0; +6,2]), mas o teste é **degenerado**: um verificador que rejeita tudo "remove" todos os falsos alarmes. Além disso, a comparação troca a família **e** a capacidade ao mesmo tempo (14B sem raciocínio contra 20B com raciocínio), então não permite concluir nada sobre erros correlacionados.

9.9 Correção

Usamos o mesmo juiz, o A3b, para todos. Uma issue só conta como corrigida se passar em tudo: veredito "válido", alerta do scanner sumiu, código compila e não surgiu alerta novo.

- **B0:** corrigiu **3 das 15** issues reais que acusou (20%; 5,4% das 56). Nos 373 achados do B0, o A3b deu 38% válido, 13% parcial e 50% inválido.
- **C2 e C3: 0 das 4**. Com tão poucas issues acusadas, a taxa não é informativa.
- **Ressalvas:** o juiz concorda pouco com os rótulos de correção dos autores (**κ = 0,28**); só 37 dos 114 trechos compilam sozinhos; o B0 gera um patch por arquivo, e o pipeline, um por achado.
- Patches desnecessários (para falsos alarmes conhecidos): 14 no C2, 2 no C3.

Conclusão: não há evidência de que verificar antes de corrigir melhore a correção.

9.10 Sensibilidade

Cada célula é recall / taxa de falsos alarmes:

Variante	B0	B1	C1 = C3	C2
Principal (regra estrita)	26,8 / 27,1	19,6 / 15,3	7,1 / 2,4	7,1 / 2,4
S1: só linha ±3, sem CWE	67,9 / 44,7	55,4 / 34,1	8,9 / 3,5	8,9 / 3,5
S2: "uncertain" aceito	26,8 / 27,1	19,6 / 15,3	8,9 / 2,4	7,1 / 2,4
S3: só segurança (54 issues)	27,8 / 27,1	20,4 / 15,3	7,4 / 2,4	7,4 / 2,4
S4: 104 arquivos fora do piloto	25,5 / 25,3	19,1 / 13,9	8,5 / 2,5	8,5 / 2,5

Variante	B0	B1	C1 = C3	C2
S5: sem as 2 extrações falhas do B0	26,8 / 27,7	19,6 / 15,7	7,1 / 2,4	7,1 / 2,4
S6: arquivo + CWE, sem linha	42,9 / 40,0	28,6 / 23,5	8,9 / 4,7	8,9 / 3,5

A conclusão qualitativa não muda em nenhuma variante. A S1 mostra quanto a regra estrita mede a CWE escolhida: o detector usa CWE-78 (injeção de comando) em quase tudo, então, ignorando a CWE, o recall do B1 sobe de 19,6% para 55,4%. Isso não muda a comparação entre configurações, porque todas usam o mesmo detector.

10. Como interpretar: a redução valeu a pena?

Ponto de operação. Quase todo detector pode trocar recall por falsos alarmes: ficando mais rigoroso, dá menos alarmes e detecta menos. Um sistema realmente melhor fica **acima** da curva dessa troca, e não só anda sobre ela. O verificador andou sobre a curva: deslocou o ponto de operação.

A conta absoluta. Sobre os locais rotulados, o C3 evitou 11 falsos alarmes e perdeu 7 issues reais. Isso só compensa se perder uma vulnerabilidade custar menos que revisar 11 ÷ 7 ≈ **1,6** alarmes falsos. Em segurança, deixar passar uma vulnerabilidade costuma custar muito mais.

As métricas combinadas concordam. O F1 cai 14,6 p.p. (intervalo de −26,0 a −4,1, significativo). Até o F0,5, que favorece a precisão, cai de 36,2% para 25,0%. O MCC vai de 0,06 para 0,12, com intervalo incluindo 0. O C3 só supera o B1 no Fβ quando β < 0,28, isto é, quando a precisão vale mais de **3,6 vezes** o recall.

O ganho real é de ruído. Arquivos negativos sinalizados caem de 54,5% para 9,1%, e os achados fora do gabarito, de 202 para 16. Se esses 186 removidos fossem quase todos falsos, a troca seria bem melhor, mas sem um anotador não sabemos.

11. Limites do estudo

- **Gabarito:** mistura riscos de padrão (um `strlen`, um `printf` que o scanner marca) com vulnerabilidades exploráveis, e o verificador procura explorabilidade. Das 11 issues reais que o B1 acusou, o A3a confirmou 4, refutou 6 e ficou incerto em 1. Os argumentos das refutações parecem plausíveis, então parte da "perda" pode ser rejeição correta; sem anotador, não dá para afirmar.
- **Tipo de falso alarme:** os 85 são só falsos alarmes de scanner. As alucinações livres do modelo ficam fora do gabarito.
- **Amostra pequena:** 56 reais e 85 falsos; o C3 acusa 4 e 2. Os intervalos são largos.
- **Regra de correspondência:** concorda pouco com os rótulos "detectado" dos autores (κ de 0,08 a 0,27, porque esses rótulos são lenientes, cerca de 80% "sim"). As sensibilidades S1 e S6 medem quanto o resultado depende dela.
- **Modelos e execução:** os modelos são menores que os do artigo; a execução principal é única, e só B1 e C3 foram repetidos.
- **Contaminação:** o DevGPT, o artigo e os dados são públicos; os modelos podem ter visto os rótulos no treino.

12. Conclusões e próximos passos

1. O artigo mede só recall relativo; o falso positivo, que é o problema central, não foi medido, e a auditoria encontrou inconsistências.
2. Num experimento controlado, o verificador adversarial reduziu muito os falsos alarmes e o ruído, mas derrubou o recall junto. A premissa de que mais um agente resolve o falso positivo sem perder detecção não se confirmou.
3. O verificador separa bem achados de scanner, mas não os achados do detector. O efeito dele depende do modelo que verifica.
4. Os números absolutos dependem muito do modelo; as comparações internas, com o mesmo modelo em todas as configurações, continuam válidas.

Contribuições: a auditoria do artigo; um pipeline reprodutível a custo zero (prompts congelados, cache, 158 testes, diário de decisões); e um jeito de medir precisão e recall usando só os rótulos que já existiam.

Próximos passos (não feitos): um anotador humano para os achados fora do gabarito; um verificador mais forte, para ver se perde menos issues reais; uma variante que separe família de capacidade do verificador; e um detector maior, para errar menos a CWE.

13. Perguntas para testar o entendimento

1. **Por que a "acurácia" de ~80% do artigo não diz nada sobre falso positivo?** Porque é issues detectadas ÷ 56: só olha para as vulnerabilidades conhecidas. Um modelo que acusasse tudo teria 100%.
2. **O que é um "falso alarme conhecido"?** Um local que um scanner marcou e que os autores examinaram e julgaram seguro. São 85.
3. **Por que C1 e C3 têm os mesmos números de detecção?** Nos dois, quem decide quais achados ficam é o A3a; as etapas a mais do C3 só tratam da correção.
4. **Por que as ferramentas ficam só no A3b?** Porque o gabarito veio dos scanners: mostrá-los ao detector ou ao verificador vazaria o gabarito. No A3b a detecção já foi decidida.
5. **A precisão do C3 subiu para 66,7%. Por que isso não confirma a H1?** Porque se apoia em 6 locais, o intervalo vai de 25% a 100%, a diferença para o B1 não é significativa e o recall caiu 12,5 p.p.
6. **O que quer dizer "o verificador não discrimina"?** Que ele não rejeita mais falsos do que reais numa proporção distinguível do acaso: manteve 4 de 11 reais e 2 de 13 falsos ($p = 0{,}36$).
7. **Por que a H5 é "degenerada"?** Porque o qwen como verificador rejeitou tudo; e a troca mudou família e capacidade ao mesmo tempo.
8. **O recall baixo é culpa do modelo pequeno?** Em parte: com o mesmo prompt, o qwen acusa 16 a 21 p.p. menos que os modelos do artigo. Mas isso não explica a H1, que usa o mesmo detector nas duas configurações.
9. **Por que a regra estrita "derruba" o recall do B1 de 55,4% para 19,6%?** Porque o detector rotula quase tudo como CWE-78; na linha certa, mas com a CWE errada, o achado não conta.
10. **Qual é o ganho real do verificador?** Menos ruído: de 54,5% para 9,1% dos arquivos negativos sinalizados.

14. Glossário

Termo	Significado
Ablação	Experimento que troca ou remove uma peça para medir o efeito dela (o C3-mf)
Achado	Uma acusação do sistema: arquivo, linhas, CWE
Bootstrap	Reamostragem com reposição para estimar a incerteza de uma métrica
Calibração	Teste do A3a sobre os achados dos scanners, antes da execução principal
Confirmatória / exploratória	Análise planejada antes para testar uma hipótese / análise feita para entender os dados
CWE	Catálogo de tipos de fraqueza de software, mantido pela MITRE
DevGPT	Conjunto público de conversas de desenvolvedores com o ChatGPT
Fora do gabarito	Achado que não casa com nenhum local rotulado; não é julgado
Issue real	Vulnerabilidade confirmada pelos autores (uma das 56)
κ (kappa)	Concordância entre avaliadores descontando o acaso
McNemar	Teste pareado que olha só os casos em que duas configurações discordam
MCC	Correlação entre o que foi acusado e o que é real; 0 = acaso
Ponto de operação	O par (recall, taxa de falsos alarmes) em que o sistema funciona
p.p.	Pontos percentuais: diferença entre duas porcentagens
Quantização	Guardar os pesos do modelo com menos bits para caber na GPU
Regra estrita / só-linha	Correspondência com CWE compatível / só pela linha (S1)
Verificador adversarial	Agente cujo papel é tentar refutar cada achado

15. Onde encontrar cada coisa

O quê	Onde
Relatório final, com todas as tabelas	`relatorio/RELATORIO.pdf`
Plano original (hipóteses, especificação dos agentes)	`PLANO_EXECUCAO.md`
Diário de decisões, hashes dos prompts, horas de GPU	`pipeline/DIARIO.md`
Auditoria do artigo	`pipeline/results/auditoria_artigo.md`
Métricas e testes completos	`pipeline/results/f7_resultados.md`
Prompts dos agentes	`pipeline/prompts/`
Código dos agentes e do pipeline	`pipeline/src/agents/`, `pipeline/src/runner.py`
Como executar o experimento	`README.md`